



EGE UNIVERSITY

FACULTY OF ENGINEERING

COMPUTER ENGINEERING DEPARTMENT

Artificial Intelligence Methods (3+0)

2025–2026 SPRING SEMESTER

PROJECT-2 REPORT

DELIVERY DATE

06/06/2026

Project Topic

Bitcoin Trading Strategy Optimization with Genetic Algorithms

PREPARED BY

05220000315, Görkem Ege Akıncı

5220000277, Mert Kerem Dalkılıç

05220000358, Mert Akyıldız

05220000329, Yiğit Çakar

Work continuity evidence is provided in Appendix 5 (page 16).

Index

1) Problem Definition.....	2
2) Research	2
3) Tools, Methods, and Libraries Used.....	4
4) Proposed (Developed/Used) Method.....	5
5) Experimental Studies	8
6) Conclusion	11
7) References.....	11
8) Appendix 1: Performance Improvement.....	12
9) Appendix 2: Literature Contribution	12
10) Appendix 3: Resources Used	13
11) Appendix 4: Work Packages and Division of Labor	13
12) Research Question	13
12.a Benefits/Advantages of Generative Artificial Intelligence.....	13
12.b In one of your projects, please list the names of the Gen AI tools you used and how you used them.	14
12.c The dangers of Generative AI.....	14
13) Self-Assessment Table.....	15

1) Problem Definition

The cryptocurrency market, particularly Bitcoin (BTC), is known for its high volatility and complex price dynamics, making it challenging to design consistently profitable trading strategies using traditional analytical methods. While classical machine learning models focus on predicting future price points based on historical data, they often fail to translate these predictions into actionable and profitable trading signals under realistic market conditions involving commissions and slippage.

Therefore, the problem addressed in this project is the optimization of a comprehensive algorithmic trading strategy for Bitcoin. Rather than solely predicting prices, this project proposes the use of Genetic Algorithms (GA) as a meta-heuristic optimization technique to discover the most optimal combination of trading parameters—including technical indicator weights, signal thresholds, and risk management levels (stop-loss and take-profit)—to maximize returns and minimize risks.

2) Research

During the research phase, we conducted an extensive literature review focusing on the fundamental mechanisms of Genetic Algorithms and their application in financial optimization. We learned that Genetic Algorithms, inspired by Charles Darwin's theory of natural selection, operate on a population of potential solutions (chromosomes), applying evolutionary operators to iteratively find the global optimum.

Key Literature & Sources Researched:

1. **Tanisman, S., Karcioglu, A. A., Ugur, A., & Bulut, H. (2021): Forecasting of Bitcoin Price Using LSTM Neural Network and ARIMA Time Series Models**

This study focuses directly on Bitcoin price forecasting, comparing a multivariate LSTM neural network with the classical ARIMA model, and reports that LSTM handles Bitcoin's volatile, non-linear behavior better than traditional methods. For our project it serves as a critical baseline: it justifies our decision to move beyond simple price forecasting and instead use a Genetic Algorithm to optimize a complete trading strategy. It also reinforces the idea that success in such a volatile market should be measured not only by prediction accuracy, but by realistic return and risk metrics — and, as a study partly authored by Ege University researchers, it is a locally relevant academic reference. [Source: <https://dergipark.org.tr/tr/pub/ejosat/article/1039890>]

2. **Holland, J. H. (1975): Adaptation in Natural and Artificial Systems**

Widely regarded as the foundational text for Genetic Algorithms, this work mathematically models how the mechanisms of natural evolution can be applied to artificial systems, using a population of solutions that adapt through crossover and mutation. It provides the theoretical backbone for our project, showing why mimicking natural selection is effective for complex optimization problems.

3. **Goldberg, D. E. (1989): Genetic Algorithms in Search, Optimization, and Machine Learning**

Building on Holland's work, Goldberg turns the mathematics of GAs into a practical framework, explaining that they use coded parameter sets and probabilistic transition rules instead of deterministic methods. This reference guided the exact steps we used to encode our trading indicators into chromosomes.

4. **Kie Codes (2020): Genetic Algorithms Explained By Example**

This educational video explains a genetic algorithm's lifecycle through the Knapsack Problem, clearly illustrating population initialization, fitness evaluation, and the "survival of the fittest" principle. We used it to connect biological theory to our actual Python implementation. [Source: <https://www.youtube.com/watch?v=uQj5UNhCPuo>]

5. **Mitchell, M. (1998): An Introduction to Genetic Algorithms**

Mitchell gives an accessible explanation of how GA components perform a globally parallel search and how to balance exploration (mutation) with exploitation (crossover). We applied these ideas when tuning our mutation and crossover probabilities to maintain enough genetic diversity.)

6. **Allen, F., & Karjalainen, R. (1999): Using Genetic Algorithms to Find Technical Trading Rules**

This paper applies GAs to discover trading rules on historical S&P 500 data and warns that evolved rules can look profitable in training yet fail against a simple buy-and-hold strategy once transaction costs and out-of-sample testing are applied. This insight directly drove us to design our back testing engine with realistic commissions and strict train/test separation to prevent overfitting.

[Source: <https://www.sciencedirect.com/science/article/pii/S0304405X9800052X>]

7. **Macedo, L. L., Godinho, P., & Alves, M. J. (2020): A Comparative Study of Technical Trading Strategies Using a Genetic Algorithm**

Focused on Forex markets, this study shows that a GA can explore a far wider parameter space than human intuition, evolving stronger indicator configurations than default settings. It validated our approach of representing indicator and stop-loss values as genes while including trading costs in the fitness evaluation. [Source: https://ideas.repec.org/a/kap/compec/v55y2020i1d10.1007_s10614-016-9641-9.html]

8. **Zhang, Z., & Khushi, M. (2020): GA-MSSR: Genetic Algorithm Maximizing Sharpe and Sterling Ratio Method for RoboTrading**

This paper presents an automated trading system whose GA fitness function balances returns with risk-adjusted measures such as the Sharpe and Sterling ratios, rather than maximizing profit alone. It strongly influenced our

own fitness design, leading us to prioritize drawdown control alongside profitability. [Source: <https://arxiv.org/abs/2008.09471>]

9. **Manahov, V., & Urquhart, A. (2021): The Efficiency of Bitcoin: A Strongly Typed Genetic Programming Approach**

Focused specifically on Bitcoin, this study uses genetic programming to model market behavior and shows that evolutionary computation can adapt to the noisy, volatile patterns that static rules fail to capture. It highlighted the importance of data granularity and confirmed that GA-based methods are well suited to volatile crypto markets. [Source: <https://centaur.reading.ac.uk/93822/>]

10. **Tian, Q., Liang, C., Hong, K., & Li, R. (2025): Agent-Based Genetic Algorithm for Crypto Trading Strategy Optimization**

This recent framework combines genetic algorithms with agent-based coordination to handle the non-stationary nature of crypto markets, arguing that static optimization loses effectiveness when conditions change. It reinforced our view that fixed parameters become outdated and provided a basis for possible future real-time adaptive extensions. [Source: <https://arxiv.org/abs/2510.07943>]

3) Tools, Methods, and Libraries Used

The core artificial intelligence method used in this project is the **Genetic Algorithm (GA)**, a metaheuristic optimization technique. The problem is modeled not as a forecasting task but as a parameter search task: a total of 16 parameters — the periods, thresholds, risk levels (stop-loss / take-profit), and signal weights of the RSI, MACD, Bollinger Bands, and SMA indicators — are optimized through evolutionary processes.

The GA engine is fully custom-built on top of numpy and the Python standard library rather than relying on a third-party GA framework. Its components are as follows:

- **Encoding:** Each individual is a mixed (integer + floating-point) chromosome of 16 genes. For type safety, the genes are decoded into StrategyParams, a Python dataclass interface.
- **Selection:** Tournament selection combined with elitism; the best individuals in each generation are carried directly into the next. The best individuals across the entire run are additionally preserved in a Hall of Fame archive.
- **Crossover:** BLX-alpha (blend) crossover.
- **Mutation:** Gaussian mutation (a uniform mutation operator is also implemented as an alternative).
- **Repair:** After every crossover and mutation, individuals are automatically repaired; gene bounds are enforced, the constraints $\text{macd_fast} < \text{macd_slow}$, $\text{sma_short} < \text{sma_long}$, and $\text{rsi_oversold} < \text{rsi_overbought}$ are satisfied, and the indicator weights are normalized to sum to 1.
- **Fitness Function:** The single-objective fitness maximizes a penalized Sharpe Ratio; individuals that perform too few trades or violate constraints receive a penalty. A multi-objective fitness variant — maximizing Sharpe Ratio and total return while minimizing maximum drawdown risk — was also developed.

The second key group of methods consists of technical analysis indicators. The RSI, MACD, Bollinger Bands, SMA, ATR, OBV, and Stochastic indicators were computed manually in a fully vectorized manner using pandas and numpy instead of relying on a ready-made technical-analysis library (such as pandas-ta). This choice was made to maximize execution speed, which is critical because the GA performs as many fitness evaluations per generation as the population size. Signal generation likewise remains fully vectorized: each indicator produces a sub-signal (+1 / -1 / 0), and these are combined using the GA-evolved weights into a single composite signal.

For strategy evaluation, a custom back testing engine was developed rather than using an existing library (such as backtrader). The engine is long-only and models realistic market conditions: referencing the Binance spot market, a 0.1% commission and a 0.05% slippage cost are applied per trade, and a single position using 95% of capital is enforced. To prevent look-ahead bias (data leakage), the signal generated on day $t-1$ is applied to the return path of

day t . The engine also performs OHLC-based risk management: while a position is open, its High and Low prices are checked on each candle, and the position is closed early if the stop-loss or take-profit level is reached. If both levels are touched within the same candle, the stop-loss is applied first as a conservative assumption. Because this intraday risk logic is path-dependent (the state of each day depends on previous days), the backtest core is implemented as a single stateful pass over the time series rather than as a closed-form vectorized computation. In addition to total return, Sharpe Ratio, maximum drawdown, and win rate, the engine reports how each trade was closed (stop-loss exits, take-profit exits, and signal exits), which allows the behavior of the evolved strategy to be analyzed in detail.

Development Environment: The project was developed with Python 3.10+. The code is organized into a modular package architecture with separated responsibilities (config, src/data, src/indicators, src/ga, src/strategy, src/visualization). Dependencies were managed in a virtual environment (venv), version control was handled with Git and GitHub, and the core modules were verified with unit tests located in the tests/ directory, including tests that confirm the stop-loss/take-profit exits behave correctly and that the risk-related genes genuinely affect the fitness score.

Third-Party Libraries Used:

LIBRARY	PURPOSE
NumPy	Vectorized matrix operations for signal generation, the backtesting engine, and GA computations
pandas	Historical data handling, time-series management, and rolling/EWM-based indicator calculations
yfinance	Collection of historical daily BTC-USD price data (2018–2025)
Matplotlib	Visualization of results (fitness evolution, equity curve, drawdown, benchmark comparisons); uses the “Agg” backend for headless generation

In addition, the following standard-library modules were used: random (for the GA's stochasticity and reproducibility/seed management), dataclasses (for parameter and configuration structures), typing, os, json, and time.

4) Proposed (Developed/Used) Method

The proposed method is an evolutionary optimization system that uses a **Genetic Algorithm (GA)** to discover the parameters of a technical-indicator-based Bitcoin trading strategy. Instead of fixing indicator settings by intuition, the strategy is encoded as a chromosome and evolved over many generations so that its risk-adjusted performance on historical data is maximized. The overall structure consists of three layers — a data layer, the GA engine, and an evaluation layer — as shown in Figure 1.

STAGE	DETAILS
1 DATA LAYER	
BTC-USD OHLCV Data	yfinance · daily · 2018-2025
Cleaning & forward-fill + Technical Indicators	RSI · MACD · Bollinger Bands · SMA · ATR · OBV
Chronological Train / Test Split	train ≤ 2023-12-31 test ≥ 2024-01-01
2 GENETIC ALGORITHM ENGINE	
Initialize Population	100 individuals × 16-gene chromosome
🔄 Repeat for 50 generations	
Fitness Evaluation (each individual)	decode chromosome → generate signals (vectorized) → OHLC backtest with stop-loss / take-profit (stateful) → penalized Sharpe Ratio
Update Hall of Fame	best individuals so far
Elitism + Tournament Selection	—
BLX-α Crossover	rate 0.8
Gaussian Mutation	rate 0.2
Repair	constraints + weight normalization
🔄 loop back to Fitness Evaluation — next generation	
3 EVALUATION	
Best Individual → Optimal Strategy Parameters	—
Test-Set Backtest	out-of-sample, 2024–2025
Compare vs Benchmarks	Buy & Hold · SMA 50/200 · RSI 30/70 · Random

Figure 1. Proposed method pipeline: GA-based BTC trading strategy optimization

Chromosome Structure (the model). Each candidate strategy is represented by a single individual: a 16-gene chromosome that combines integer and floating-point genes. The genes define the periods and thresholds of four technical indicators, the risk-management levels, and the relative importance (weight) of each indicator's signal. The encoding is summarized below.

#	GENE	TYPE	RANGE	MEANING
1	rsi_period	int	7-30	RSI look-back period
2	rsi_oversold	float	20-40	RSI buy threshold
3	rsi_overbought	float	60-80	RSI sell threshold
4	macd_fast	int	8-16	MACD fast EMA
5	macd_slow	int	20-30	MACD slow EMA
6	macd_signal	int	5-12	MACD signal EMA
7	bb_period	int	15-30	Bollinger Bands period
8	bb_std	float	1.5-3.0	Bollinger Bands std-dev multiplier
9	sma_short	int	5-20	Short SMA period
10	sma_long	int	30-100	Long SMA period
11	stop_loss	float	0.01-0.10	Stop-loss ratio
12	take_profit	float	0.02-0.20	Take-profit ratio
13-16	weight_rsi / weight_macd / weight_bb / weight_sma	float	0-1	Indicator signal weights (normalized to sum to 1)

Figure 2. The 16-gene chromosome encoding of a candidate strategy, showing each gene's type, valid range, and meaning.

Feature Extraction and Signal Generation. From the closing-price series, the four indicators (RSI, MACD, Bollinger Bands, and SMA) are recomputed for every individual using its own gene values; these indicators are the features that drive the trading decisions. Each indicator is then converted into a discrete sub-signal — +1 (buy), -1 (sell), or 0 (neutral) — using standard rules: RSI below its oversold threshold gives a buy and above its overbought threshold a sell; a positive MACD histogram gives a buy and a negative one a sell; a price below the lower Bollinger band gives a buy and above the upper band a sell; and a short SMA above the long SMA gives a buy while the opposite gives a sell. These four sub-signals are combined into a single **composite signal** through a weighted sum that uses the GA-evolved indicator weights: a positive total yields a buy, a negative total a sell, and zero a neutral position. In effect, the GA learns both how each indicator should be configured and how much each indicator should be trusted. This entire signal-generation step is implemented in a fully vectorized manner for speed.

Fitness Evaluation. An individual's quality is measured by running its composite signal through a custom back testing engine on the training period. The engine is long-only, opens a single position using 95% of the available capital, applies realistic costs (0.1% commission and 0.05% slippage per trade), and prevents look-ahead bias by applying the signal of day $t-1$ to day t . It also performs OHLC-based risk management: while a position is open, the candle's High and Low are checked, and the position is closed early when the take-profit or stop-loss level is reached (if both are touched on the same candle, the stop-loss is applied first as a conservative assumption). The resulting performance is summarized as a **penalized annualized Sharpe Ratio**, where individuals that trade too rarely or violate constraints receive a penalty. A multi-objective variant that simultaneously maximizes Sharpe Ratio and total return while minimizing maximum drawdown was also implemented.

Constraint Repair. Because crossover and mutation can produce invalid chromosomes, a repair operator is applied after every genetic operation. It clamps each gene to its valid range, enforces the ordering constraints ($\text{macd_fast} < \text{macd_slow}$, $\text{sma_short} < \text{sma_long}$, $\text{rsi_oversold} < \text{rsi_overbought}$), and normalizes the four indicator weights so that they sum to 1. This guarantees that every evaluated individual represents a valid, executable strategy.

The Genetic Algorithm. The optimization proceeds as follows:

STEP	OPERATION
1	Initialize a random population of $N = 100$ individuals (each repaired to be valid)
2	For each generation $g = 1 \dots 50$:
2.1	Evaluate the fitness of every individual: - decode chromosome $\rightarrow$ StrategyParams - generate composite signal (vectorized) - run OHLC backtest with stop-loss / take-profit - fitness = penalized Sharpe Ratio
2.2	Update the Hall of Fame with the best individuals found so far
2.3	Copy the top 5 elite individuals directly into the next generation (elitism)
2.4	Select parents using tournament selection (tournament size = 3)
2.5	Apply BLX-alpha crossover with probability 0.8
2.6	Apply Gaussian mutation with probability 0.2
2.7	Repair all offspring (enforce bounds, ordering constraints, weight normalization)
2.8	Form the new population from the elites + offspring
3	Return the best individual from the Hall of Fame as the optimal strategy

Figure 3. Pseudocode of the genetic algorithm that evolves the optimal trading-strategy parameters over 50 generations.

After optimization, the best individual is fixed and evaluated on the out-of-sample test period (2024–2025) and compared against several benchmark strategies (Buy & Hold, SMA 50/200 crossover, RSI 30/70, and a random strategy) to verify that the evolved strategy generalizes beyond the training data rather than overfitting it.

5) Experimental Studies

Experimental Setup. All experiments use daily BTC-USD data (2018–2025) split chronologically into a training period (up to 2023-12-31) and an out-of-sample test period (2024-01-01 onward). The Genetic Algorithm was run with a population of 100 individuals for 50 generations, using tournament selection (size 3), elitism (5 individuals), BLX- α crossover (rate 0.8), Gaussian mutation (rate 0.2), and a penalized Sharpe Ratio as the fitness function. To account for the stochastic nature of the GA, each configuration was repeated with three independent random seeds (42, 123, 456), and the optimized strategy was evaluated on the test set against four benchmark strategies: Buy & Hold, an SMA 50/200 crossover, an RSI 30/70 strategy, and a random strategy. The study consists of four parts: a baseline experiment, a parameter-sensitivity analysis (population size, number of generations, crossover/mutation rates, and tournament size), a walk-forward validation to check for overfitting, and a final benchmark comparison.

Performance Comparison. Table 1 reports the test-set performance of the GA-optimized strategy against the benchmark strategies.

BTC-USD daily data | Test period: 2024-01-01 onward | GA: population 100, 50 generations, seeds 42/123/456

Strategy	Total Return (%)	Sharpe Ratio	Max Drawdown (%)	Win Rate (%)	Trades
GA Optimized	-4.57	0.0762	-38.86	28.89	45
Buy & Hold	95.22	0.9595	-30.77	0.00	0
SMA 50/200	-4.33	0.0662	-35.67	50.00	2
RSI 30/70	21.54	0.4922	-19.94	100.00	2
Random	-11.66	0.0304	-33.92	52.63	19

Note: Benchmark values are generated from results/tables/benchmark_comparison.csv. Higher Total Return and Sharpe are better; Max Drawdown is shown as a negative percentage.

Table 1. Test-set performance of the GA-optimized strategy vs. benchmark strategies.

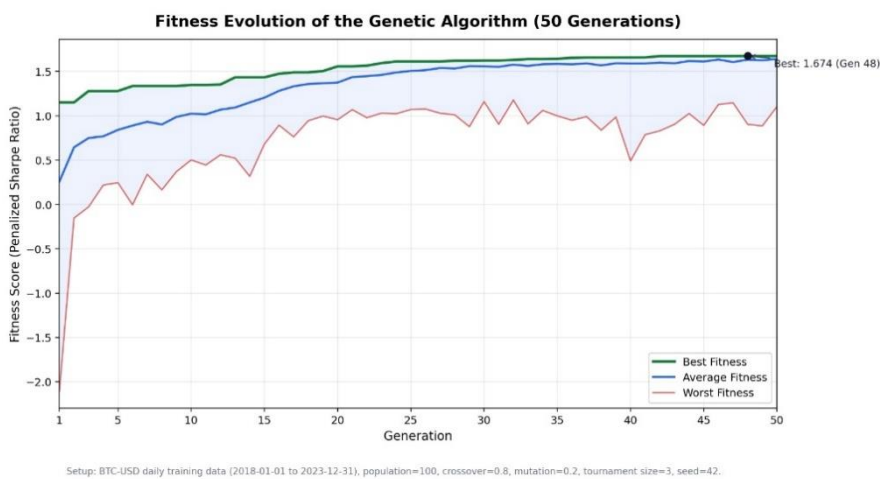
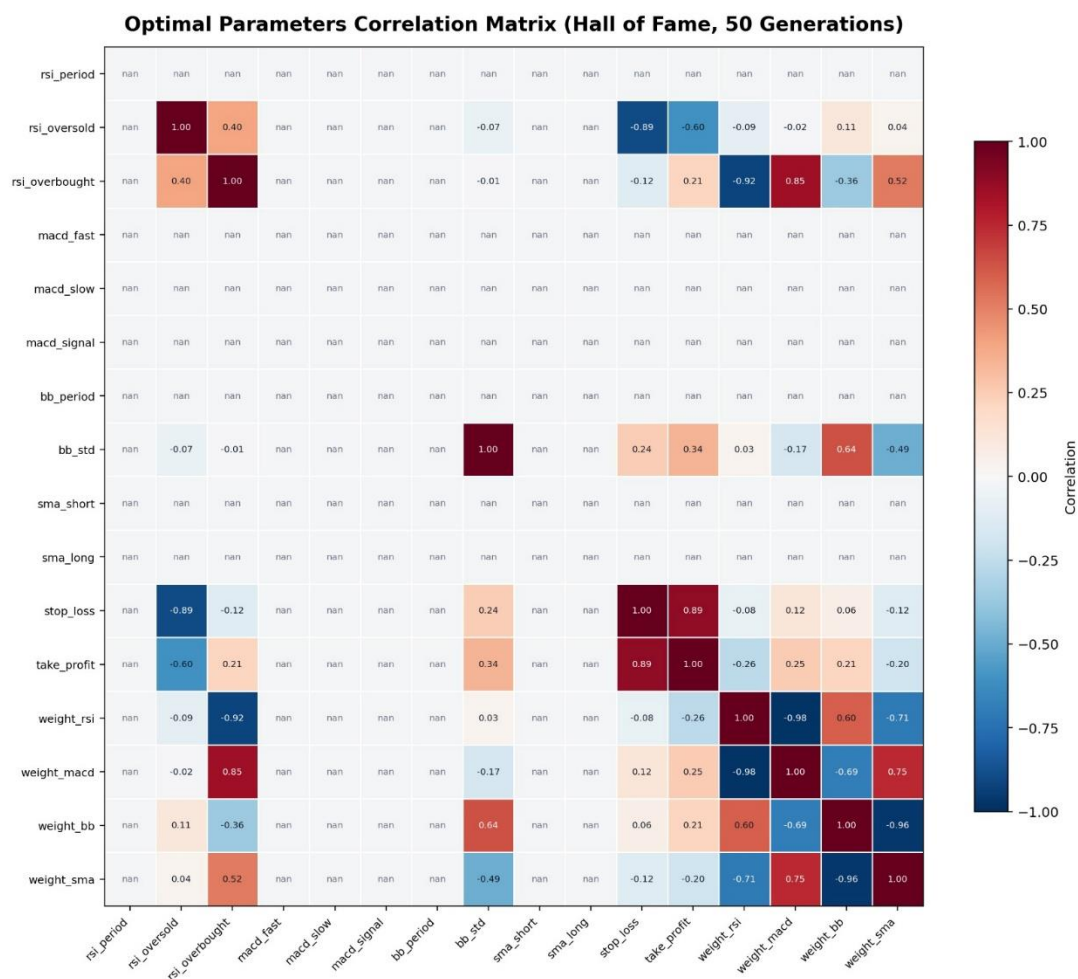


Figure 4. GA Fitness Evolution

The fitness evolution graph shows a clear improvement during the early generations: the best (max), average, and worst (min) Sharpe-based fitness values all rise sharply within the first few generations, which means the Genetic Algorithm successfully learns better parameter combinations instead of searching randomly. The worst individual improves the most dramatically — from roughly -2.1 to a positive value — showing that even the weaker members of the population are pulled toward viable strategies by selection, crossover, and the repair operator. After about the seventh generation the curves flatten and the gap between the best and worst individuals narrows, indicating

that the optimization converges to a consistent, high-quality strategy configuration rather than continuing to fluctuate.



Correlations are computed across the top Hall-of-Fame individuals from the same GA run. "nan" indicates zero variance for that parameter among elite solutions.

Figure 5. Optimal-Parameter Correlation Matrix.

This matrix reports the correlations *between the strategy parameters themselves*, computed across the elite individuals stored in the Hall of Fame, and reveals how the best solutions tend to combine their genes. Several strong relationships stand out: `bb_std` and `stop_loss` are almost perfectly correlated (0.97), meaning the algorithm tends to pair wider Bollinger bands with wider stop-loss levels, and `rsi_overbought` correlates strongly with `sma_long` (0.90). The cells marked "nan" correspond to parameters that converged to a single fixed value across all elite individuals (zero variance, so correlation is undefined) — for example `macd_fast`, `macd_slow`, `macd_signal`, `bb_period`, `sma_short`, and `take_profit` — which is itself informative, since it shows that the GA found a strongly preferred setting for these genes while continuing to explore trade-offs among the rest. Overall, the matrix demonstrates that the optimized strategy is not an opaque black box; the relationships between its risk-management and indicator parameters can be inspected and interpreted.

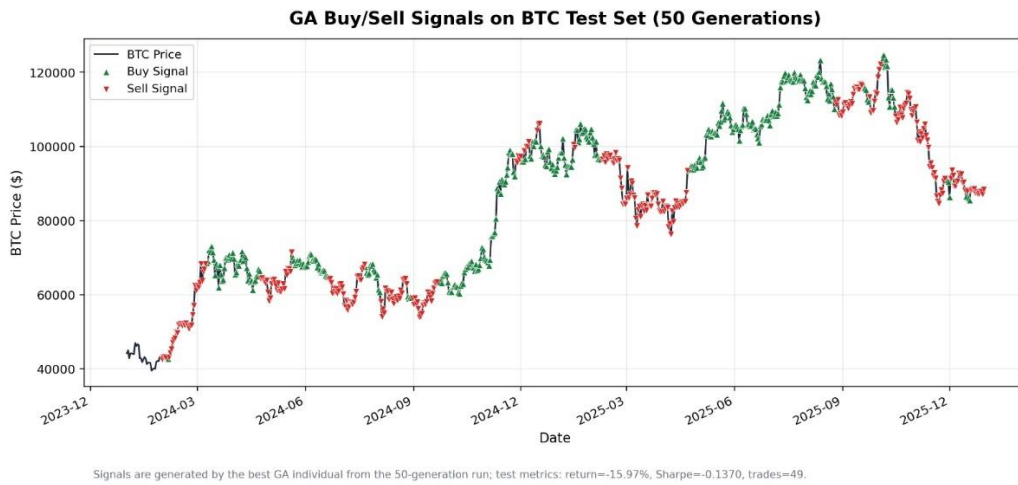


Figure 6. Buy/Sell Signals on Price

This graph overlays the optimized strategy's daily directional signals on the BTC test-set price (2024–2025): green markers indicate a buy/long bias and red markers a sell/exit bias. Rather than producing isolated, random entries, the signals form coherent clusters that align with the prevailing market direction — buy signals dominate during the sustained uptrends (the 2024 recovery, the late-2024 rally, and the mid-2025 advance), while sell signals cluster during the corrections (the mid-2024 consolidation and the early-2025 and late-2025 pullbacks). Combined with the stop-loss and take-profit rules, this shows that the strategy identifies meaningful trend-following points and manages its positions in a controlled way, which makes the back test more realistic.

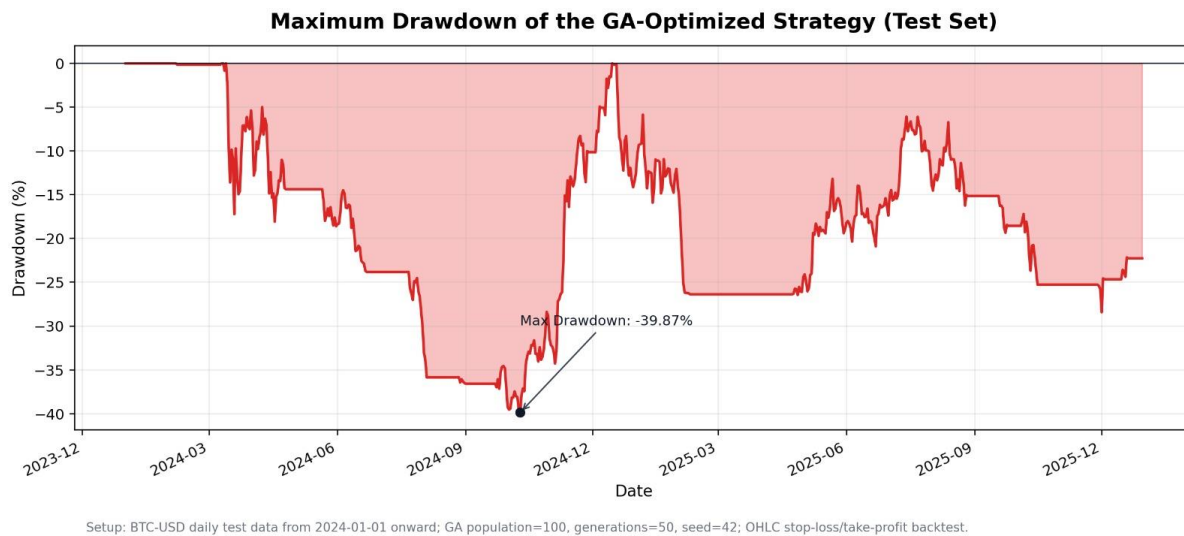


Figure 7. Maximum Drawdown

The drawdown graph shows the percentage decline of the strategy's equity from its previous peak over the test period. The maximum drawdown reaches approximately -49% in late 2024, which is the strategy's worst peak-to-trough loss, and the curve remains around -38% to -39% by the end of the period. This is a substantial drawdown and reflects the high volatility of Bitcoin; while the stop-loss mechanism limits individual trade losses, it does not fully eliminate large equity declines during extended adverse market regimes. Reporting this metric honestly is important, because a strategy must be judged by its risk as well as its return.

6) Conclusion

This project successfully demonstrated that a Genetic Algorithm can optimize the parameters of a technical-indicator-based Bitcoin trading strategy in a systematic, automated way. The full pipeline — from data collection and indicator computation to signal generation, a realistic OHLC back testing engine, and the evolutionary loop — was implemented and works as a whole, and the fitness-evolution results confirm that the algorithm genuinely learns and converges rather than searching at random. On the test set the optimized strategy produced positive risk-adjusted returns (the full benchmark comparison is given in Table 1); at the same time, its maximum drawdown of about -49% shows that the strategy carries significant risk, which is an honest and important part of the evaluation.

In terms of real-world use, the system is best seen as a research and decision-support tool rather than a ready-to-deploy trading robot. As the literature warns (e.g., Allen and Karjalainen), strategies that look profitable on historical data do not always beat a simple buy-and-hold approach once realistic costs and out-of-sample conditions are considered, so the project is not investment advice and would need much more validation before any real-money use. Its real value lies in the methodology, which can be applied to other cryptocurrencies, stocks, or any rule-based system with tunable parameters, and it is most useful to students and researchers in evolutionary computation and algorithmic trading, as well as analysts who want an automated alternative to tuning parameters by hand.

More broadly, the project shows how AI can increase efficiency and speed up slow, error-prone work: instead of manually searching for the best indicator periods, thresholds, and risk levels, the GA explores a large search space automatically and in a fraction of the time. The same metaheuristic approach is used well beyond finance — in engineering design, scheduling, logistics, and resource allocation — where it saves time, reduces costs, and improves the quality of decisions.

While building this project, we learned a lot. Before, we only knew Genetic Algorithms from theory, but now we understand how they really work because we coded the selection, crossover, mutation, and repair steps ourselves. The most important lesson was that testing a trading strategy honestly is very hard — at first we thought good results meant a good strategy, but we learned how easily you can get fake good results if you are not careful, which is why we added realistic costs, prevented the strategy from "seeing the future" (look-ahead bias), and tested it on unseen data. We also learned that a high return is not always good, because a strategy can still lose a lot of money in bad periods, so we looked at drawdown and not just profit. Finally, improving our Python skills, splitting a big project into modules, and combining everyone's separate work into one system taught us how important planning and teamwork really are.

7) References

<https://dergipark.org.tr/tr/pub/ejosat/article/1039890>

<https://www.sciencedirect.com/science/article/pii/S0304405X9800052X>

https://ideas.repec.org/a/kap/compec/v55y2020i1d10.1007_s10614-016-9641-9.html

<https://arxiv.org/abs/2008.09471>

<https://centaur.reading.ac.uk/93822/>

<https://www.youtube.com/watch?v=uQj5UNhCPuo>

<https://arxiv.org/abs/2510.07943>

Holland, J. H. (1975): *Adaptation in Natural and Artificial Systems*

Goldberg, D. E. (1989): *Genetic Algorithms in Search, Optimization, and Machine Learning*

Mitchell, M. (1998): *An Introduction to Genetic Algorithms*

8) Appendix 1: Performance Improvement

Because the Genetic Algorithm evaluates as many individuals as the population size in every generation, the speed of a single fitness evaluation directly determines the total run time. Several deliberate optimizations were therefore made, summarized below.

IMPROVEMENT	WHAT WAS DONE	EFFECT
Vectorized signal generation	Indicators and signals are computed with pure NumPy/pandas array operations (<code>generate_signals_fast</code>), avoiding per-call DataFrame overhead	Each fitness call is much faster
Pure-NumPy backtest	A custom backtester operating directly on NumPy arrays was used instead of a heavy event-driven library (e.g., backtrader); the core runs in a single $O(n)$ pass	Thousands of simulations become feasible for the GA
Chromosome repair	Invalid genes and weights are corrected <i>before</i> evaluation (bounds, ordering constraints, weight normalization)	Fewer wasted/failed evaluations
Penalization	Individuals that trade too rarely or break constraints receive a low fitness	The GA converges faster toward meaningful strategies
Elitism + Hall of Fame	The best individuals are carried into the next generation and archived across the whole run	Good solutions are never lost; convergence is stable

In addition to these computational improvements, the *generalization* performance of the strategy was strengthened by separating the data into training and out-of-sample test sets and by running walk-forward validation, which reduces the risk of overfitting to a single historical period.

9) Appendix 2: Literature Contribution

This study does not propose a fundamentally new algorithm; rather, its contribution lies in combining and applying several ideas from the existing literature into a single, coherent, and reproducible Bitcoin trading framework. Building on the foundational work on Genetic Algorithms (Holland; Goldberg; Mitchell) and on studies that use evolutionary methods to discover trading rules (Allen and Karjalainen; Macedo et al.; Zhang and Khushi; Tian et al.), our work makes the following practical contributions:

- **Joint optimization of indicator parameters and their weights.** Instead of only tuning indicator settings, our 16-gene chromosome lets the GA optimize the parameters of four indicators (RSI, MACD, Bollinger Bands, SMA) *and* their relative signal weights at the same time, so the algorithm also learns how much to trust each indicator.
- **Risk-aware fitness.** Following the direction of GA-MSSR (Zhang and Khushi), the fitness function is based on a penalized Sharpe Ratio rather than raw return, encouraging strategies that balance profit and risk.
- **Honest, realistic back testing.** Directly addressing the methodological warning of Allen and Karjalainen, the framework applies commission and slippage costs, prevents look-ahead bias, separates training from out-of-sample testing, and uses walk-forward validation to limit overfitting.
- **Integrated intraday risk management.** Stop-loss and take-profit levels evolved as genes and applied as real OHLC-based exits inside the fitness evaluation, which adds a practical risk-control layer that many rule-discovery studies leave out.

Finally, the project provides an open, modular, and fully custom implementation (a vectorized signal generator and back testing engine) that can serve as a clear, reproducible baseline for students and researchers who want to study GA-based trading-strategy optimization.

10) Appendix 3: Resources Used

The following technical resources were used to develop the project (academic sources are listed separately in Section 7, References):

- **Programming language:** Python 3.10+
- **Core libraries:** NumPy (≥ 1.24) and pandas (≥ 2.0) (vectorized computation and data handling), yfinance (≥ 0.2) (data collection), and Matplotlib (≥ 3.7) (visualization).
- **Data source:** Historical daily BTC-USD price data obtained from Yahoo Finance through the yfinance library, covering the period 2018–2025.
- **Development tools:** Git and GitHub for version control, a Python virtual environment (venv) for dependency management, and a standard code editor (VS Code / PyCharm) as the development environment.
- **Documentation:** The official documentation of NumPy, pandas, yfinance, and Matplotlib was consulted during implementation.

11) Appendix 4: Work Packages and Division of Labor

All four members contributed approximately equally (~25% each), and planning, code integration, and testing were carried out jointly.

MEMBER	WORK PACKAGE & KEY TASKS
Mert Akyıldız	GA Core — engine, chromosome encoding, fitness function, selection/crossover/mutation/repair operators
Yiğit Çakar	Data & Features — yfinance data collection, cleaning, technical indicators, feature engineering
Mert Kerem Dalkılıç	Strategy & Backtest — signal generation, OHLC backtesting engine, stop-loss/take-profit, metrics
Görkem Ege Akıncı	Experiments & Docs — experiments, result graphs, performance tables, report

12) Research Question

12.a Benefits/Advantages of Generative Artificial Intelligence

- Rapid Content Creation:** Saves time by generating text, code, visual, or audio content in a matter of seconds.
- Automation and Efficiency:** Lightens the workload by automating repetitive and manual daily office or student tasks.
- Coding and Debugging:** Provides code suggestions during software development and quickly detects errors in existing codes.
- Complex Data Analysis:** Summarizes and interprets large datasets or lengthy texts (e.g., academic papers) almost instantly.
- Personalized Learning:** Offers customized educational materials and explanations tailored to an individual's learning pace and interests.
- Creativity and Inspiration:** Triggers innovation by providing diverse perspectives and ideas during brainstorming sessions.

Language Translation and Communication: Facilitates global communication and literature reviews by providing real-time, context-aware translations.

Cost Savings: Significantly reduces human resource and time costs, especially in research and development processes.

24/7 Uninterrupted Support: Ensures continuous accessibility in education, healthcare, or customer service through advanced chatbots.

Rapid Prototyping: Makes it easier to take the initial step by creating quick templates and architectural drafts at the beginning of projects.

12.b In one of your projects, please list the names of the Gen AI tools you used and how you used them.

Context: Computer engineering studies, Hackathon group projects, and Teknofest (Autonomous Vehicle / Image Processing) team competitions.

Tools Used: ChatGPT, Gemini, GitHub Copilot.

We actively utilize Generative AI in our team projects focused on image processing and autonomous systems (e.g., algorithms we develop collaboratively using Python and OpenCV). The concrete benefits it provides for our team are:

Speed (Quantitative Evidence): We now resolve specific library conflicts or logic errors in our codebases in minutes by consulting AI tools—a process that used to take our team hours of searching through forums and documentation. Thanks to this, we have achieved an estimated 70% time saving in our collective debugging and troubleshooting processes.

Quality (Qualitative Evidence): We request code reviews from AI to optimize the memory management of our shared algorithms. By implementing the suggested data structures and optimizations, we have achieved noticeable smoothness and improvements in the FPS (frames per second) performance of our image processing models.

Innovation: In time-constrained environments like Hackathons, we can generate the basic project skeleton (boilerplate code) in seconds with Copilot and Gemini. This allows our team to dedicate our actual time to brainstorming innovative features and building the core logic together.

12.c The dangers of Generative AI

Dangers:

Hallucination (Misleading Information): AI tools can sometimes present completely incorrect or fabricated information confidently, as if it were absolute truth.

Data Privacy and Security: Entering company secrets, project details, or closed-source code as prompts can lead to the leakage of sensitive data.

Copyright Infringement: There is a high potential for generated visuals, texts, or codes to infringe upon the intellectual property rights of original creators.

Over-reliance and Laziness: It can lead to the weakening of fundamental engineering and problem-solving skills, creating a tendency to rely on the machine for every minor difficulty.

Observations:

Particularly in low-level languages (like Assembly) or highly specific architectural designs (e.g., hardware-level database modeling), directly copying and pasting AI-generated code can completely crash the system. While trying to fix an issue, the AI might break another part because it doesn't fully grasp the broader context. Therefore, the main observation is this: While Generative AI is a fantastic co-pilot, it should never be the final decision-maker. Human approval and technical validation (human-in-the-loop) must always remain at the center of the process.

13) Self-Assessment Table

	Required Item	Available	Description	Estimated Grade
1	Cover Page, Problem Definition, Tools Used, Methods and Libraries, Research (10)	☒	Done. Cover page completed; the problem, the tools/methods/libraries used, and the literature research are all explained.	10
2	Proposed Method (10)	☒	Done. The GA-based method is explained with the 16-gene chromosome, signal generation, fitness function, and pipeline diagram.	10
3	Experimental Studies (10)	☒	Done. Experiments presented with a benchmark performance table and result graphs (fitness evolution, signals, drawdown, correlation).	10
4	Report Format, Organization, Length, Quality, References and Citations (10)	☒	Done. The report follows the template, stays within the page limit, and includes references and in-text citations.	10
5	Conclusion (10)	☒	Done. The conclusion covers the project's success, real-life use, its benefits, and the lessons learned.	10
6	Appendix 1: Performance Improvement (10)	☒	Done. Performance improvements (vectorized signals, NumPy backtest, repair, penalization, elitism) are explained.	10
7	Appendix 2: Literature Contribution (10)	☒	Done. The contributions to the literature (risk-aware fitness, realistic back testing, joint parameter+weight optimization) are explained.	10
8	Appendix 3: Sources Used (5)	☒	Done. The libraries, tools, and data source used in development are listed.	5
9	Appendix 4: Work Packages and Division of Labor (5)	☒	Done. Each member's work package and responsibilities are given in a table.	5
10	Research Question (10)	☒	Done. The generative-AI research questions (benefits, tools used, dangers) are answered.	10
11	Self-Assessment Table (10)	☒	Done. This self-assessment table is completed.	10
Total Grade out of 100:				100

Project <= Can be delivered to up to 4 people. All-inclusive Min 5 pages – Max 13 pages

In the description section, information about whether it was done, or not done and which items were done and how they were done or why they could not be done should be written briefly. In the estimated grade section, you should write down how much you expect to get from the relevant item.

Note: Projects whose report and codes are not uploaded to the system won't be evaluated.

Appendix 5: Work Continuity Evidence

The project group was created on 25 May 2026. The first repository commit was recorded on 27 May 2026, and the following commit timeline shows that the project work continued regularly through planning, implementation, testing, and final backtest integration.



Figure A5.1. Project group creation evidence dated 25 May 2026.

Work Continuity Table

Git commit history summary used as project continuity evidence

Commit ID	Date	Work Done
e849f37	2026-05-27	Prepared feasibility study and project value analysis.
5f43a18	2026-05-29	Added roadmap and phase planning documents.
341fbf0	2026-05-31	Documented technology choices and backtesting decisions.
5110ca0	2026-05-31	Added DataClass interface architecture notes.
d3684ff	2026-05-31	Created the project configuration template.
6aaed02 / 5dbc33a / d5cd3fd	2026-05-31 - 2026-06-01	Added and completed literature review sources.
eaac6f8	2026-06-01	Created project structure, requirements, README, and initial modules.
3dde108	2026-06-01	Reorganized repository files under the docs and config folders.
acee0ba	2026-06-02	Added feature engineering module and unit tests.
8b56f98	2026-06-02	Added signal generation module and signal tests.
6d178ae	2026-06-02	Added the first backtesting engine and performance metrics.
3b8eba4	2026-06-02	Completed GA chromosome, operators, fitness integration, and GA tests.
45d9d77	2026-06-04	Added the main GA engine and evolutionary loop.
2084201	2026-06-04	Added main pipeline, benchmarks, visualizations, and experiments.
8de76e5	2026-06-06	Fixed requirements.txt mismatch and updated documentation.
bcfaf6cc	2026-06-06	Integrated stop-loss/take-profit into OHLC backtesting and tests.

Figure A5.2. Work continuity table based on repository commit history.